Apply filters to SQL queries

Project description

As a security professional working in a large organization, I discovered some potential security issues that involve login attempts and employee machines.

I was tasked with examining the organization's data in their **employees** and **log_in_attempts** tables. I used SQL filters to retrieve records from different datasets and investigate the potential security issues.

Retrieve after hours failed login attempts

A potential security incident occurred after business hours (18:00). In order to investigate this incident, I performed a filtered query in the **`log_in_attempts`** table.

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE login_time > '18:00' AND success = FALSE;
+----------+----------+------------+------------+---------+----------------+---------+
| event_id | username | login_date | login_time | country | ip_address     | success |
+----------+----------+------------+------------+---------+----------------+---------+
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12 |       0 |
|       18 | pwashing | 2022-05-11 | 19:28:50   | US      | 192.168.66.142 |       0 |
|       20 | tshah    | 2022-05-12 | 18:56:36   | MEXICO  | 192.168.109.50 |       0 |
|       28 | aestrada | 2022-05-09 | 19:28:12   | MEXICO  | 192.168.27.57  |       0 |
|       34 | drosas   | 2022-05-11 | 21:02:04   | US      | 192.168.45.93  |       0 |
|       42 | cgriffin | 2022-05-09 | 23:04:05   | US      | 192.168.4.157  |       0 |
|       52 | cjackson | 2022-05-10 | 22:07:07   | CAN     | 192.168.58.57  |       0 |
|       69 | wjaffrey | 2022-05-11 | 19:55:15   | USA     | 192.168.100.17 |       0 |
```

The query in the screenshot shows `` `SELECT * FROM log_in_attempts WHERE login_time > '18:00' AND success = FALSE;` ``. Here is a breakdown: First part, "`SELECT * FROM log_in_attempts`" to select all the columns in the `log_in_attempts` table. Second part, "`WHERE login_time > '18:00' AND success = FALSE;`" this is a conditional statement (where clause) that specifies the login time to be after the business hours and the login attempt must fail. In the screenshot the table returned 0s in the success column; the 0 indicates a false value in binary. This query returns all login attempts that failed after the business hours.

Retrieve login attempts on specific dates

A suspicious event occurred on 2022-05-09. To investigate this event, I reviewed all login attempts that occurred on this day and the day before.

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |       1 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |       1 |
|        4 | dkot     | 2022-05-08 | 02:00:39   | USA     | 192.168.178.71  |       0 |
|        8 | bisles   | 2022-05-08 | 01:30:17   | US      | 192.168.119.173 |       0 |
|       12 | dkot     | 2022-05-08 | 09:11:34   | USA     | 192.168.100.158 |       1 |
|       15 | lyamamot | 2022-05-09 | 17:17:26   | USA     | 192.168.183.51  |       0 |
|       24 | arusso   | 2022-05-09 | 06:49:39   | MEXICO  | 192.168.171.192 |       1 |
|       25 | sbaelish | 2022-05-09 | 07:04:02   | US      | 192.168.33.137  |       1 |
```

This query returns all the login attempts on `2022-05-09` and `2022-05-08`. First, I selected all the columns within the log_in_attempts table. Second, I used the `OR` operator to filter the `WHERE` clause. **`OR`** returns all rows that apply to one of two conditions in order to retrieve all the login attempts in two days (the 9th and 8th of May 2022).

Retrieve login attempts outside of Mexico

The security team determined that this suspicious activity didn't originate in Mexico. Therefore, I used SQL filters to investigate further into this incident. However, I was notified that the country column in the database contains values of both **MEX** and **MEXICO.**

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE NOT country LIKE 'MEX%';
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |       1 |
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12  |       0 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |       1 |
|        4 | dkot     | 2022-05-08 | 02:00:39   | USA     | 192.168.178.71  |       0 |
|        5 | jrafael  | 2022-05-11 | 03:05:59   | CANADA  | 192.168.86.232  |       0 |
|        7 | eraab    | 2022-05-11 | 01:45:14   | CAN     | 192.168.170.243 |       1 |
|        8 | bisles   | 2022-05-08 | 01:30:17   | US      | 192.168.119.173 |       0 |
|       10 | jrafael  | 2022-05-12 | 09:33:19   | CANADA  | 192.168.228.221 |       0 |
```

The query "`SELECT * FROM log_in_attempts WHERE NOT country LIKE 'MEX%';`" returns all the login attempts that did not originate in Mexico. The `WHERE` clause contains two operators: the `NOT` operator, which returns all countries that aren't Mexico, and the `LIKE` operator, which is used instead of the equal operator "`=`" whenever there is a wildcard in the comparison. The percentage "`%`," which accepts any characters after "`MEX`." For example, this condition applies to "`MEX`" and "`MEXICO`."

Retrieve employees in Marketing

The security team wants to perform security updates on specific employee machines in the marketing department. I am responsible for getting information on these employee machines and will need to query the **`employees`** table.

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE department = 'Marketing' AND office LIKE 'East%';
+-------------+--------------+----------+------------+----------+
| employee_id | device_id    | username | department | office   |
+-------------+--------------+----------+------------+----------+
|        1000 | a320b137c219 | elarson  | Marketing  | East-170 |
|        1052 | a192b174c940 | jdarosa  | Marketing  | East-195 |
|        1075 | x573y883z772 | fbautist | Marketing  | East-267 |
|        1088 | k8651965m233 | rgosh    | Marketing  | East-157 |
|        1103 | NULL         | randerss | Marketing  | East-460 |
|        1156 | a184b775c707 | dellery  | Marketing  | East-417 |
|        1163 | h679i515j339 | cwilliam | Marketing  | East-216 |
+-------------+--------------+----------+------------+----------+
```

This query returns all employees that are in the marketing department and work in the east offices. It starts first with the `SELECT` statement for selecting all employees before filtering the results with `WHERE` alongside `AND`, `LIKE`, and the wildcard "`%`".

Retrieve employees in Finance or Sales

A security update for machines for employees in the Sales and Finance departments needs to be performed. I used a query that filters in SQL all employees in sales or finance.

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE department = 'Finance' OR department = 'Sales';
+-------------+--------------+----------+------------+-------------+
| employee_id | device_id    | username | department | office      |
+-------------+--------------+----------+------------+-------------+
|        1003 | d394e816f943 | sgilmore | Finance    | South-153   |
|        1007 | h174i497j413 | wjaffrey | Finance    | North-406   |
|        1008 | i858j583k571 | abernard | Finance    | South-170   |
|        1009 | NULL         | lrodriqu | Sales      | South-134   |
|        1010 | k242l212m542 | jlansky  | Finance    | South-109   |
|        1011 | l748m120n401 | drosas   | Sales      | South-292   |
|        1015 | p611q262r945 | jsoto    | Finance    | North-271   |
|        1017 | r550s824t230 | jclark   | Finance    | North-188   |
```

The first part of the screenshot is my query, and the second part is a portion of the output. This query returns all employees in the Finance and Sales departments. First, I selected all the data in the employees table. Then I applied a condition with `WHERE` and `OR` to return only employees in sales and finance.

Retrieve all employees not in IT

Your team needs to make one more update to employee machines. The employees who are in the information technology department already had this update, but employees in all other departments need it. I used an SQL query to achieve this.

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE NOT department = 'Information Technology';
+-------------+--------------+----------+----------------+-------------+
| employee_id | device_id    | username | department     | office      |
+-------------+--------------+----------+----------------+-------------+
|        1000 | a320b137c219 | elarson  | Marketing      | East-170    |
|        1001 | b239c825d303 | bmoreno  | Marketing      | Central-276 |
|        1002 | c116d593e558 | tshah    | Human Resources | North-434   |
|        1003 | d394e816f943 | sgilmore | Finance        | South-153   |
|        1004 | e218f877g788 | eraab    | Human Resources | South-127   |
|        1005 | f551g340h864 | gesparza | Human Resources | South-366   |
|        1007 | h174i497j413 | wjaffrey | Finance        | North-406   |
|        1008 | i858j583k571 | abernard | Finance        | South-170   |
```

This query returns all employees who are not in IT. First, I select all data from the employees table. Then I added an operator `NOT` to filter the results in order to exclude the IT employees.

Summary

In this activity, I implemented SQL queries to support the organization's cybersecurity efforts. For instance, I applied filters on the logs database table in order to investigate a security incident. In addition, I supported a patch management process by identifying the machines that were missing a patch update. I accomplished all of this by leveraging operators like `AND`, `OR`, `LIKE`, and `NOT` within SQL conditions.

Written by : Ahmed Abdulqader